RESUMELENS: RÉSUMÉ–PROFILE MATCHING WITH FORMAL LANGUAGE MODELS

Regular expressions → finite-state transducers → finite automata → context-free DSL

Authors: Isabella Candado, Alejandro Murillo **Affiliations:** Icesi University - Department of CIS - Computing and Discrete Structures III

Ingestion
Tools: TXT · MD · DOCX · PDF loaders
Transformation: file → text

Stage 1 - Extraction
Tools: regular expressions (re)
Transformation: text → surface forms
Class: regular languages

Stage 2 - Normalization
Tools: transducers (pyformlang)
Transformation: surface forms → 43 tokens
Class: rational relations

Projection
Tools: rank-based canonical sort
Transformation: token set → word w over ΣP

Stage 3 - Classification
Tools: one DFA per profile (pyformlang)
Transformation: word → accepted/rejected
Class: regular languages

Stage 4 - Representation
Tools: CFG in EBNF (textX)
Transformation: report → DSL → .md
Class: context-free languages

1 - Problem and context

- **Goal:** Decide whether the qualifications explicitly stated in a résumé satisfy a profile pattern. Classification, not ranking.
- **Variation:** (i) spelling skills; (ii) order skills; (iii) cv format; (iv) profile logic.
- **Question:** Can each stage use the weakest sufficient formal model, composed into one verifiable pipeline?

Profile	Groups	$\|\Sigma\|$	Skills sets	Accepted
Full Stack Developer	7	18	262,144	33,075 (12.6%)
ML Engineer	7	12	4,096	135 (3.3%)
Cloud Engineer	6	13	8,192	1,029 (12.6%)
Data Analyst	4	12	4,096	2,205 (53.8%)

Table 1. *Size of the decision problem per profile. 43 canonical skills; 94 specified spellings; 4 input formats. Each profile's space is small enough to verify exhaustively.*

2 - Stage 1: Extraction with regular expressions

- **Model:** Finite union of regular expressions (re), restricted to alternation, concatenation, repetition and one-character look-arounds (no back-references): every pattern stays regular.
- **Boundary guards:** SQL ∉ PostgreSQL, Git ∉ GitHub, JS ∉ Node.js.
- **Ambiguity:** keeps Node, Excel, ML case-sensitive; R only inside the skills section.
- **Output:** Surface forms as written; canonicalization is Stage 2.
- **Structure:** Bilingual section headers; e-mail regex; phone numbers of 9–15 digits.

Skill	Matches	Does not match
JAVASCRIPT	JS, JavaScript, java script	Node.js
SQL	SQL, sql	PostgreSQL, NoSQL, MySQL
EXCEL	Excel, EXCEL, MS Excel	Excellent, excel (verb)

Table 2. *Three of the 43 skill recognizers. The guard rules out the listed false positives.*

3 - Stage 2: Normalization with finite-state transducers

- **Goal:** Aliases → one canonical token (JS → JAVASCRIPT).
- **Model:** Deterministic Mealy FST $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F), \delta \subseteq Q \times \Sigma \times Q, \omega : \delta \to \Gamma^*$
- **Cascade:** trim → **T0** collapse separators → **T1** case folding → **T2** one machine per skill.
- **T2 = prefix tree** of the spellings S_T; token written on the first transition, so REACT ⊂ REACTJS gives one token.

$$Q_T = \mathrm{Pref}(S_T), \quad \omega_T(\varepsilon, c) = T, \quad \omega_T(x, c) = \varepsilon\ (x \neq \varepsilon), \quad F_T = S_T, \quad R_T = \{(s, T) : s \in S_T\}$$

- **Disjoint spellings** (checked at build time) make the union a function.

Tier	Q	Σ	Γ	q_0	F	$\|\delta\|$
T0 Stripping	$\{q_s, q_w, q_p\}$	letters, digits, ␣ . - _ / (67)	letters, digits, ␣ (63)	q_s	Q	201
T1 Case folding	$\{q_0\}$	letters, digits, ␣ (63)	A–Z, digits, ␣ (37)	q_0	$\{q_0\}$	63
T2 token T (example REACT)†	$\mathrm{Pref}(S_T)$ (11 states)	characters of S_T	$\{T\}$	ε	S_T	$\|Q_T\| - 1$ (10)

Table 3. *Tiers of the cascade. T2 totals: 43 machines, 94 spellings, 568 states, 525 transitions.*

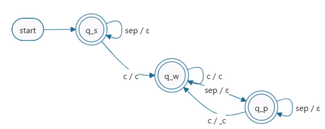


Figure 1. *Tier 0. The space is emitted lazily (q_p → q_w), which removes trailing separators without a second pass.*



Figure 2. *Token transducer for REACT (spellings REACT, REACTJS, REACT JS). REACT is accepting and also has outgoing transitions.*

4 - Canonical projection

Stage 2 gives a set S of tokens, in any order. Σ_P = tokens of profile P; rank_P = their order in its groups.

Equation $w = \pi_P(S) = \mathrm{sort}_{\mathrm{rank}_P}(S \cap \Sigma_P)$ Filter, sort, no repeats: $k!$ orders → one word.

Example {GIT, REACT, DOCKER, SQL, JAVASCRIPT} → JAVASCRIPT REACT SQL GIT

5 - Stage 3: Recognition with finite automata

- **Group rules:** *REQUIRED*: all tokens appear. *ANY_OF*: at least one. *OPTIONAL*: none required.
- **Token-level DFA (built by the code):** $|Q| = 1 + |\Sigma_P|$
- **Group-level DFA (Figures 3–6):** one state per group boundary. A comma-separated label means or; ↺ means "any further token of the group just satisfied".

Equation $M = (Q, \Sigma, \delta, q_0, F), \quad Q = \{q_0, \ldots, q_n\}, \quad F = \{q_n\}$
$\delta(q_g, t) = q_{g+1}\ \ (t \in G_g), \qquad \delta(q_{g+1}, t) = q_{g+1}\ \ (t \in G_g,\ G_g \text{ ANY_OF})$

Type of automaton

- **Partial DFA:** One start state; every transition reads one token (no ε-moves); at most one successor per token. Not an NFA, not an ε-NFA.
- Canonical order lets the state remember only the current group. Unordered input would need 2^n states.
- Token-level and group-level automata give the same verdict on projected words.

Full Stack Developer

Language → front end → back end → REST → relational DB → non-relational DB → Git

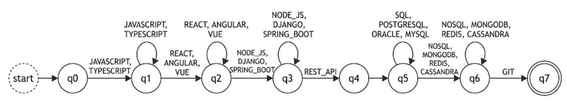


Figure 3. *Full Stack Developer. 7 groups (5 ANY_OF, 2 REQUIRED), 8 states. Needs one relational and one non-relational database. Accepts 33,075 of 262,144 skill sets.*

Machine Learning Engineer

Python → Pandas or NumPy → scikit-learn → TensorFlow or PyTorch → ML modelling → relational DB → Git

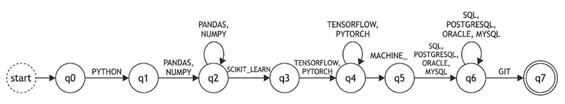


Figure 4. *Machine Learning Engineer. 7 groups (3 ANY_OF, 4 REQUIRED), 8 states. Accepts 135 of 4,096 skill sets.*

Cloud Engineer

Provider → containers → infrastructure as code → CI/CD → Linux → Git

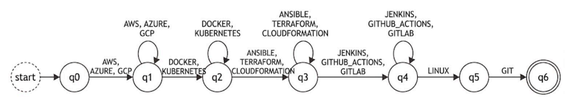


Figure 5. *Cloud Engineer. 6 groups (4 ANY_OF, 2 REQUIRED), 7 states. Accepts 1,029 of 8,192 skill sets.*

Data Analyst

Relational DB → analysis tool → visualization → analysis library

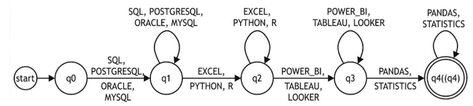


Figure 6. *Data Analyst. 4 groups (all ANY_OF), 5 states. Accepts 2,205 of 4,096 skill sets.*

6 - Stage 4: Validation and representation

- **Model:** CFG in EBNF compiled by textX (a PEG engine: ordered choice, deterministic).
- **Syntax:** Nested blocks, separated lists, regular terminals.
- **Semantics:** Profile name ∈ registry; every evaluated token is declared in skills.
- **Flow:** serialize → validate → render; invalid documents are rejected.

DSL Grammar (EBNF)

```
Candidate: 'candidate' '{'
    contact=Contact
    'education' '{' education+=STRING '}'
    'certifications' '{' certifications*=STRING '}'
    'skills' '{' skills+=Skill[','] '}'
    'profiles' '{' results+=ProfileResult '}'
'}' ;
ProfileResult: name=STRING status=Status
    'evaluated' '[' evaluated+=[Skill|SkillName][','] ']' ;
Status: 'ACCEPTED' | 'REJECTED' ;
SkillName: /[A-Z][A-Z0-9]*(_[A-Z0-9]+)*/ ;
```

Figure 7. *Excerpt of the grammar candidate.tx (the experience line and the rules Contact, Skill, Email, Phone are omitted).*

DSL Document (Example)

```
candidate {
  contact { … }
  skills { JAVASCRIPT, TYPESCRIPT, REACT, … }
  profiles {
    "Full Stack Developer" ACCEPTED evaluated [JAVASCRIPT, …, GIT]
```

Figure 8. *Excerpt of the grammar candidate.tx (the experience line and the rules Contact, Skill, Email, Phone are omitted).*

7 - Results: one résumé through the pipeline

Input - sample résumé

```
Jane Doe
jane.doe@example.com | +57 300 123 4567

## Summary
Full-stack engineer with some cloud exposure.

## Technical Skills
JS, TypeScript, React.js, Node.js, REST APIs, PostgreSQL, MongoDB, Git, Docker,
Linux, K8s, Python, Excel

## Experience
- Full-Stack Developer, ACME (2021 - 2024)
- Built REST APIs with Node.js and PostgreSQL

## Education
B.Sc. Computer Science, Universidad del Valle, 2020

## Certifications
* Scrum Master
```

(a) Stage 1 output for the sample résumé

Field	Value
Contact	Jane Doe - jane.doe@example.com - +57 300 123 4567
Entries	education 1 - experience 2 - certifications 1
Raw skills (13)	JS, TypeScript, React.js, Node.js, REST APIs, PostgreSQL, MongoDB, Git, Docker, Linux, K8s, Python, Excel

Table 4.a. *Stage 1 output.*

(b) Stage 2 trace

Raw	After T0	After T1	Token
JS	JS	JS	JAVASCRIPT
React.js	React js	REACT JS	REACT
Node.js	Node js	NODE JS	NODE_JS
REST APIs	REST APIs	REST APIS	REST_API
K8s	K8s	K8S	KUBERNETES
Excel	Excel	EXCEL	EXCEL
GitHub	GitHub	GITHUB	none (not in vocabulary)
C++	outside Σ	—	none (reported as unrecognized)

Table 4.b. *Stage 2 trace: T0 and T1 remove separators and case, T2 maps spellings to tokens. 13 forms give 13 tokens, none unrecognized. Out-of-vocabulary and out-of-alphabet input is reported, not raised.*

(c) Stage 3 traces

Profile	Projected word w	Run	Verdict
Full Stack Developer	JAVASCRIPT TYPESCRIPT REACT NODE_JS REST_API POSTGRESQL MONGODB GIT	$q_0 \to q_1 \to q_1 \to q_2 \to q_3 \to q_4 \to q_5 \to q_6 \to q_7$	✓ ACCEPTED ($q_7 \in F$)
Machine Learning Engineer	PYTHON POSTGRESQL GIT	$q_0 \to q_1$, then no transition on POSTGRESQL (waiting for PANDAS or NUMPY)	✗ REJECTED (halted in q_1)
Cloud Engineer	DOCKER KUBERNETES LINUX GIT	no transition on DOCKER from q_0 (waiting for AWS, AZURE or GCP.)	✗ REJECTED (halted in q_0)
Data Analyst	POSTGRESQL EXCEL PYTHON	$q_0 \to q_1 \to q_2 \to q_2$ (loop on PYTHON)	✗ REJECTED (ended in $q_2 \notin F$; waiting for a visualization tool)

Table 5. *Acceptance visits every group (the loop at q1 absorbs a second alternative). Rejection is an undefined transition (Machine Learning at q1, Cloud at q0) or input ending in a non-accepting state (Data Analyst at q2).*

(d) Stage 4 outcomes

Case	Kind	Line:col	Message
V1 complete document	valid	—	model returned
V2 profile name replaced by "Astronaut"	SEMANTIC	13:9	Unknown profile name: 'Astronaut'
V3 closing } of profiles removed	SYNTAX	16:2	Expected '}'
V4 education { } (empty block)	SYNTAX	7:17	Expected STRING
V5 evaluated lists an undeclared token	SEMANTIC	13:82	Unknown object "UNDECLARED" of class "Skill"

Table 6. *Real validator output. Syntax and semantic errors are reported separately; the grammar rejects empty += lists (V4) and undeclared tokens (V5).*

8 - Limitations and future work

Limitations

- **Matching:** Explicit mentions only; no semantic inference or experience-year analysis. Homonyms (Oracle, React, Pandas) and bare tokens (Node, REST) rely on form and case, not context.
- **Alphabet:** T0 and T1 cover ASCII letters, digits, and five separators; C++, C#, and accents fall outside Σ.
- **Scalability:** Each skill requires a pattern, vocabulary, and test. Sizes grow linearly, but exhaustive verification becomes impractical beyond ~25 tokens per profile.
- **Formats:** Two-column PDFs may be misordered; scans need OCR; DOCX tables are ignored.
- **Extensibility:** New profiles and skills require no grammar changes, unlike new sections; += rejects empty sections.
- **Evaluation:** No corpus testing yet; verification traces do not measure accuracy.

Future work

- **Explain rejections:** Identify where each run stops.
- **Graded matching:** Rank near misses with weighted automata.
- **External profiles:** Compile files into automata; support OPTIONAL and multi-token REQUIRED groups.
- **Robustness:** Add fuzzy transducers, Unicode, and OCR.
- **Evaluation:** Test on an annotated résumé corpus.

9 - Conclusions

- Each stage uses the weakest model that suffices: regular expressions, rational transducers, DFAs, then a CFG.
- Specifying stages separately makes them testable: verdicts agree on all 278,528 projected skill sets.
- A new profile needs a group list, plus vocabulary entries for any new skill.

Check	Scope	Result
Transducers against the reference character-level construction	94 spellings, 43 machines	94/94 spellings give their token; 3,948 cross-token and 8,970 invalid inputs are rejected
Token-level DFA, group-level DFA and direct rule check	all 278,528 projected skill sets of the four profiles	identical verdicts
Order independence of the projection	120 permutations of 5 tokens	one distinct word

10 - References

1.Hopcroft J. E., Motwani R., Ullman J. D. Introduction to Automata Theory, Languages, and Computation, 3rd ed. Pearson, 2006.
2.Mohri M. Finite-State Transducers in Language and Speech Processing. Computational Linguistics 23(2), 1997.
3.Ford B. Parsing Expression Grammars: A Recognition-Based Syntactic Foundation. POPL, 2004.
4.Dejanović I., Vaderna R., Milosavljević G., Vuković Ž. TextX: A Python tool for Domain-Specific Languages implementation. Knowledge-Based Systems 115, 2017.
5.Romero J. Pyformlang: An Educational Library for Formal Language Manipulation. SIGCSE, 2021.

11 - Repository

The following QR code contains all the information about the implementation and documentation of the project.